学习任务自查表

序号	本实验需要的背景知识	超星平台慕课视频	预习情况自查
1	利用 EasyX 库生出图形界面		□ 已完成预习
2	利用 GetAsyncKeyState()函数异步检测按键输入		□ 已完成预习
3	结构体		□ 已完成预习
4	函数定义及函数封装		□ 已完成预习
5	一维数组、结构体作为函数的输入参数		□ 已完成预习
6	结构体指针、结构体指针作为函数的输入参数		□ 已完成预习
7	模块化程序设计		□ 已完成预习
8	字符串及字符串指针		□ 已完成预习
9	界面显示的重绘更新机制		□ 已完成预习
10	Easyx 的 ExMessage 结构体定义与应用		□ 已完成预习

案例项目 1：简单的计算器

一、总体内容和要求

1. 问题描述

设计一个界面简单的计算器 ，屏幕上要有：五个按键，表示加、减、乘、除和退出功能；三个消息提示框，显示：“数据 1”、“数据 2”和“计算结果”；两个数据输入框和一个数据输出框；一个信息提示输出框，显示“点击数据 1/数据 2 后键盘输入；点击 + - * / 计算；ESC 或 退出 结束”

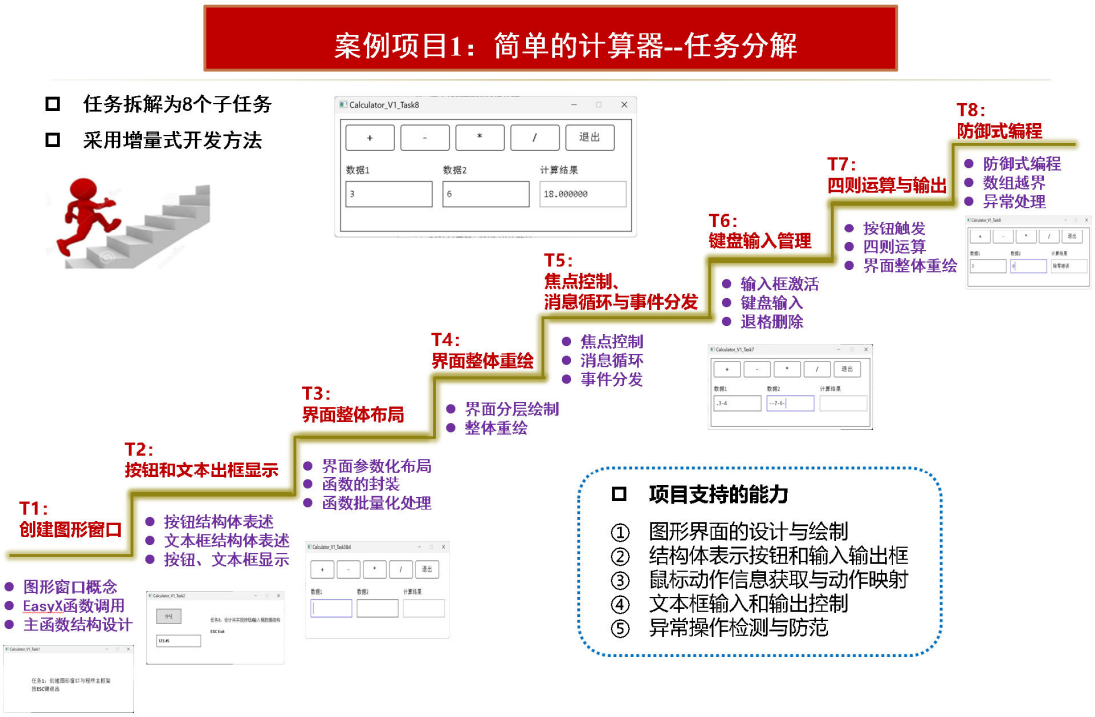
整体布局如下：

2. 能力目标

（1）掌握图形界面的设计与绘制
（2）掌握利用结构体表示按钮和输入输出框
（3）掌握鼠标动作信息的获取与动作映射
（3）掌握文本框的输入和输出控制
（4）掌握异常操作的检测与防范

3. 任务拆解

任务 1：创建图形窗口与程序主框架
任务 2：设计并实现按钮和输入/输出框
任务 3：完成项目界面布局及显示
任务 4：实现整界面重绘功能
任务 5：实现文本框焦点管理及消息循环与事件分发
任务 6：实现键盘字符输入
任务 7：实现四则运算与结果输出
任务 8：程序健壮性处理与交互优化



4. 实验环境

操作系统：Windows
开发环境：VS Studio2022 或以上
第三方库：EasyX 库 （Ver:26.1.1）

任务 1：创建图形窗口与程序主框架

1. 任务要求

（1）数据输入操作要求

无

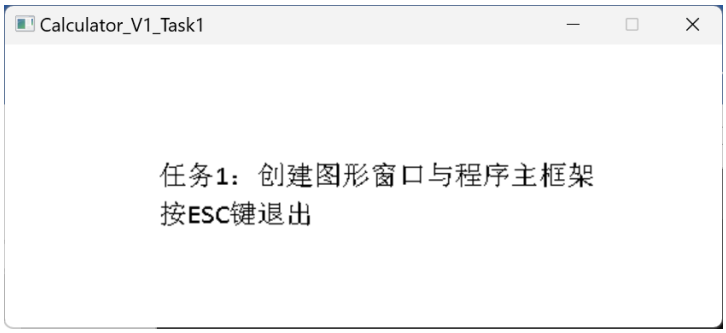
（2）数据存储和处理要求

无

（2）数据输出显示要求（即运行界面）

完成程序入口 main()函数及建立基本运行循环；使用 EasyX 函数创建图形窗口并显示文字；完成程序 ESC 或按钮退出。

程序运行结果如图所示：



2. 能力要素分析

（1）掌握图形界面设计的基本原理

（2）学会 EasyX 基本函数 initgraph()、closegraph()等的使用

（3）掌握程序的循环控制与程序退出机制

3. 实现思路

（1）算法思路

利用 while(1)语句进行程序主循环设计，在主循环中不断处理输入的各类消息，满足退出条件时调用 closegraph() 并 return 0。

（2）实现流程

Step1：在 main()中调用 initgraph(width, height)创建窗体 560x220

Step2：设置背景颜色（如白色，WHITE）

Step3：用函数 cleardevice()进行清屏操作

Step4：设置字体颜色、字体风格

Step5：显示窗体提示信息

Step6：设置主循环等待输入

Step7：退出时用 closegraph()关闭窗口

4. 问题解析

（1）为什么 GUI 界面不能“画完就结束”?

GUI 程序必须持续运行，等待用户点击/输入；如果没有主循环，程序会很快退出，窗口一闪而过，因此主程序需要一个循环机制，

```
while (1) {
    // 可利用peekmessage函数，进行非阻塞取消息
    // 区分 EM_MOUSE | EM_KEY | EM_CHAR 等多类消息，并进行相应的处理
        }
```

（2）为什么退出前要 closegraph()?

用于释放图形资源并关闭 EasyX 窗口，避免资源未正确回收。

5. 实现要点

（1）程序设计的头文件

程序要包含一系列头函数，以支持系统函数的调用，主要包括：

```
#include <graphics.h>    // 引入 EasyX 图形库：窗口、绘图、消息（ExMessage）等
#include <tchar.h>       // TCHAR / _T 宏 / _tcscpy_s / _tcslen 等：兼容 ANSI/Unicode
#include <math.h>        // 数学函数
#include <windows.h>     // Sleep、VK_ESCAPE、消息常量等
```

（2）图形窗口的创建

创建窗口使用 initgraph()函数：

功能：创建一个指定大小的图形窗口，并进入图形模式。在图形模式下，可用 easyx 提供的各种绘图函数（如 line、circle、rectangle 等）在窗口中进行绘制。initgraph 是 EasyX 图形程序的入口点，只有调用它之后才能使用其他绘图函数。

函数原型：

```
void initgraph(
int width,          // 窗口宽度
int height,         // 窗口高度
int flag = NULL     // 窗口样式，默认为 NULL
);
```

实例：

若创建一个 560x220 的图形窗口，则 initgraph(560, 220);

（3）设置背景颜色

利用 setbkcolor() 函数设置当前绘图背景色。

注意：它设置的是背景颜色，但并不会立即改变窗口背景，需要配合 cleardevice 函数

来清除屏幕并使用新设置的背景色填充，或者在其他一些操作中（例如输出文字）作为背景。

函数原型：void setbkcolor(COLORREF color);

参数 color：指定的背景颜色，可以是 RGB 函数返回的颜色值（如 RGB(255,0,0) 表示红色），也可用 EasyX 中预定义的颜色常量（如 RED、BLUE 等）。

（4）色彩的 RGB 表示

RGB 函数用于将红、绿、蓝三个颜色分量组合成一个 COLORREF 类型的颜色值，这是 Windows 图形编程中标准的颜色表示方式。

```
COLORREF RGB(
    BYTE byRed,      // 红色分量 (0-255)
    BYTE byGreen,    // 绿色分量 (0-255)
    BYTE byBlue      // 蓝色分量 (0-255)
);
// 基础颜色
COLORREF red     = RGB(255, 0, 0);     // 红色
COLORREF green   = RGB(0, 255, 0);     // 绿色
COLORREF blue    = RGB(0, 0, 255);     // 蓝色
COLORREF white   = RGB(255, 255, 255);// 白色
COLORREF black   = RGB(0, 0, 0);       // 黑色
COLORREF yellow = RGB(255, 255, 0);   // 黄色
COLORREF purple = RGB(255, 0, 255);   // 紫色
COLORREF cyan    = RGB(0, 255, 255);   // 青色
```

（5）清屏操作

函数 cleardevice() 函数是 EasyX 图形库中用于清空绘图窗口的函数，它将整个窗口用当前背景色填充，相当于清屏操作。

函数原型：void cleardevice();

（6）设置文字字体

函数 settextstyle() 是 EasyX 图形库中设置文字样式的核心函数，用来定义后续 outtextxy() 输出文字的字体大小、方向、字体类型等属性，即：先设定文字长什么样，再输出文字。

函数原型：

void settextstyle(int nHeight, int nWidth, LPCTSTR lpszFace);

参数	含义
nHeight	文字高度（单位：像素），比如 24 就表示文字高度为 24 像素（正数）；设为 0 则自适应宽度
nWidth	文字宽度（单位：像素），设为 0 表示宽度按字体比例自动适配（推荐用 0）

参数	含义
lpszFace	字体名称（字符串），比如 _T("Consolas")、_T("宋体")、_T("黑体")

示例：

```
settextstyle(24, 0, _T("Consolas"));
```

（7）文字输出

函数 outtextxy() 是 EasyX 图形库中在指定坐标位置输出文字。

函数原型：

void outtextxy(int x, int y, const TCHAR *str); //适配宽字符

void outtextxy(int x, int y, const char *str); //适配多字节字符

注意事项：

该函数使用前提是必须先创建图形窗口，可提前设置文字颜色/字体样式。

（8）Windows 系统字符编码

Windows 系统有 ANSI 和 Unicode 两种字符编码，区别如下：

特性	ANSI 编码	Unicode 编码（Windows 主流用 UTF-16）
存储大小	英文/数字占 1 字节，中文占 2 字节	几乎所有字符（含中文 / 英文）占 2 字节
覆盖范围	仅支持单种语言（如 GB2312 = 简体中文）	全球所有语言（中文、英文、日文等）
兼容性	旧程序常用，不同语言编码不通用	现代 Windows 系统默认，跨语言通用
在代码中的表现	对应 char 类型（1 字节）	对应 wchar_t 类型（2 字节）

TCHAR 是兼容宏 —— 编译时会根据项目设置自动转为 char（多字节）或 wchar_t（宽字符），_T("") 是配套的字符串宏，保证编码一致。

TCHAR 和 _T("") 是配套使用的编码兼容方案，保证跨编译环境正常显示中文；

示例：

```
outtextxy(120, 90, _T("任务1：创建图形窗口与程序主框架"));
outtextxy(120, 120, _T("按ESC键退出"));
```

（9）异步检测按键输入

函数 GetAsyncKeyState() 是 Windows 系统提供的 API 函数（非 EasyX 专属）。

功能：异步检测某个按键当前的状态（按下/未按下），即“问系统：这个键现在有没有被按住？”

优势：不依赖输入缓存、响应快，特别适合图形界面/游戏中实时检测按键。

定义：SHORT GetAsyncKeyState(int vKey);

用法：

1. 参数 vKey：要检测的按键（用系统宏表示）

Windows 定义了大量按键宏：

按键	对应的宏	说明
ESC 键	VK_ESCAPE	退出程序常用
回车键	VK_RETURN	确认 / 提交操作
方向键上	VK_UP	向上移动 / 选择
方向键下	VK_DOWN	向下移动 / 选择
方向键左	VK_LEFT	向左移动
方向键右	VK_RIGHT	向右移动
空格键	VK_SPACE	暂停 / 确认
字母键 A-Z	0x41-0x5A	比如 0x41 是 A 键
数字键 0-9	0x30-0x39	比如 0x30 是 0 键

2. 返回值：按键状态（重点看最高位）

返回值是 SHORT（短整型），只需要关注**最高位（第 15 位）**：

- ✧ 如果最高位是 1 → 按键**正在被按下**；
- ✧ 如果最高位是 0 → 按键**未被按下**。

实际使用时，会用 & 0x8000 提取最高位（0x8000 是二进制 1000 0000 0000 0000）

示例：若要检测是否是 ESC 按键按下

```
if (GetAsyncKeyState(VK_ESCAPE) & 0x8000)
{
 // ESC 键被按下，执行退出/对应操作
}
```

（10）主循环中不直接忙等，配合 Sleep(10) 降低 CPU 占用。

（11）关闭程序回收资源

函数 closegraph()：关闭图形窗口并释放相关资源，结束图形模式，恢复正常文本模式，是 EasyX 图形程序的"结束"函数。

函数原型：void closegraph();

6.参考代码

```
#include <graphics.h>
#include <conio.h>
#include <tchar.h>
// 新增头文件：用于 GetAsyncKeyState 函数
#include <windows.h>

int main( ) {
// 1) 创建图形窗口
initgraph(560, 220);

// 2) 简单显示内容
setbkcolor(WHITE);
cleardevice();
settextcolor(BLACK);
settextstyle(24, 0, _T("Consolas"));

outtextxy(120, 90, _T("任务1：创建图形窗口与程序主框架"));
outtextxy(120, 120, _T("按ESC键退出"));

// 3) 主循环：按 ESC 退出（修复核心）
while (1) {
// 检测 ESC 键是否被按下（VK_ESCAPE 是 ESC 键的系统宏）
if (GetAsyncKeyState(VK_ESCAPE) & 0x8000) {
break; // 按下 ESC 则退出循环
}

Sleep(10); // 加10ms延迟，降低CPU占用
 }

// 4) 关闭窗口
closegraph();
return 0;
 }
}
```

任务 2：设计并实现按钮和输入/输出框

1. 任务要求

（1）数据输入操作要求

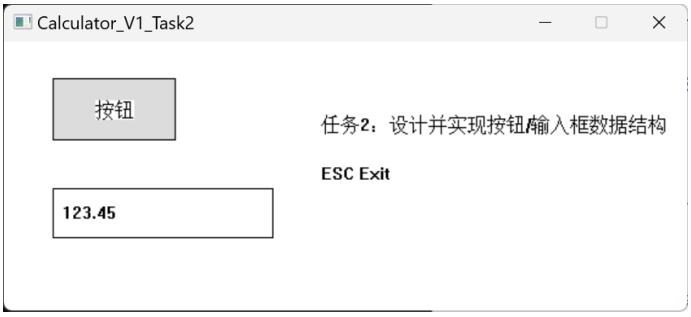
无

（2）数据存储和处理要求

利用结构体表示按钮 Button 和文本框 FieldRect 信息

（2）数据输出显示要求（即运行界面）

在任务 1 的基础上，新增输出一个按钮并标识按钮文字；输出一个文本框并在其中显示文字。界面如下图所示



2. 能力要素分析

1) 掌握利用结构体建模按钮（把界面对象抽象为数据）
2) 掌握利用结构体建模输入/输出框
3) 掌握利用字符数组来表示字符串缓冲区

3. 实现思路

（1）数据结构

利用结构体来表征按钮 Button 和文本框的具体参数，定义如下：

```
typedef struct {
    int x, y, w, h;
    TCHAR text[16];
    COLORREF color;
} Button;
```

```
typedef struct {
    int x, y, w, h;
} FieldRect;
```

利用字符数组来表示文本框输出的信息，

```
static TCHAR demoText[32] = _T("123.45");
```

（2）算法思路

利用结构体变量赋值来设置具体的按钮和文本框参数；按钮和文本框都是利用

EasyX 函数画出来的。

（3）实现流程

Step1：在 main()中调用 initgraph(width, height)创建窗体 560x220

Step2：设置背景颜色（如白色，WHITE）

Step3：用函数 cleardevice()进行清屏操作

Step4：设置字体颜色、字体风格

Step5：按钮和文本框结构体变量赋值，设置具体的参数

Step6：调用 EasyX 函数绘制按钮和文本框

Step7：在文本框输出信息

Step8：设置主循环等待输入

Step9：退出时用 closegraph()关闭窗口

4. 问题解析

1) 按钮和输入框采用不同的结构体表示

由于按钮除了位置，还有显示文字与颜色；输入框只需位置即可。分开设计更清晰、职责单一。

2) 按钮和输入/输出框的生成

利用 EasyX 库，按钮和输入/输出框等都是通过画图方式直接在指定位置画出来的，因此，再画之前要给出各个要素的设置，详见实现要点。

5. 实现要点

（1）用结构体表示按钮 Button

```
// Button：描述一个按钮
// x,y,w,h：按钮左上角坐标、宽、高
// text：按钮显示文字（最多 15 个字符 + 结束符）
// color：填充颜色
typedef struct { int x, y, w, h; TCHAR text[16]; COLORREF color; } Button;
```

若定义五个按钮，可实例化五个结构体，如下：

```
static Button opBtns[5];        // 5 个按钮：+ - * / 退出
```

（2）static 关键字的含义

系统保留关键字，特点如下：

作用	通俗解释
存储位置 / 生命周期	变量不再存在于 “临时栈区”，而是在“全局静态区”，程序运行全程都存在（直到程序结束）
作用域限制	如果在函数内定义，变量只在当前函数内可见；如果在全局定义，只在当前文件内可见

（3）用结构体表示输入/输出提示框

```
// FieldRect：描述一个矩形区域（输入框/输出框）
// x,y,w,h：左上角坐标、宽、高
typedef struct { int x, y, w, h; } FieldRect;
```

若定义“数据 1”、“数据 2”、“计算结果”三个显示框，则有：

```
static FieldRect field1, field2, fieldR; // 三个框：数据 1、数据 2、计算结果
```

（4）用字符数组表示显示框的数据

示例如下：

```
// 输入与输出字符串缓冲区（TCHAR 支持 Unicode）
// buf1/buf2：两个输入框内容
// bufR：结果输出框内容
static TCHAR buf1[32] = _T("");
static TCHAR buf2[32] = _T("");
static TCHAR bufR[64] = _T("");

static TCHAR demoText[32] = _T("123.45");
```

（5）按钮 Button 的绘制与现实

按钮的绘制可以分为四个具体步骤：

A. 设置按钮实体颜色； setfillcolor()
B. 画出按钮实体； solidrectangle()
C. 设置按钮边框的颜色；setlinecolor()
D. 画出按钮边框； rectangle()

把按钮的实体颜色和边框颜色分别控制，可以灵活控制按钮属性。

示例：

```
 // 绘制按钮
 setfillcolor(demoBtn.color);
 solidrectangle(demoBtn.x, demoBtn.y, demoBtn.x + demoBtn.w, demoBtn.y +
demoBtn.h);
 // 给按钮也加上黑色边框（可选，让按钮样式统一）
 setlinecolor(BLACK);
 rectangle(demoBtn.x, demoBtn.y, demoBtn.x + demoBtn.w, demoBtn.y + demoBtn.h);
```

```
// 给按钮上添加文字
outtextxy(demoBtn.x + 35, demoBtn.y + 18, demoBtn.text);
```

（6）输入/输出框的绘制

与按钮相似，设置颜色→画边框→输出文字

示例：

```
// 绘制输入框（核心修改：设置线条颜色为黑色）
setlinecolor(BLACK); // 显式指定边框线条为黑色
rectangle(demoField.x, demoField.y, demoField.x + demoField.w, demoField.y +
demoField.h);
outtextxy(demoField.x + 8, demoField.y + 12, demoText);
```

任务 3：完成项目界面布局及显示

1. 任务要求

（1）数据输入操作要求

无

（2）数据存储和处理要求

利用结构体表示五个按钮、三个文本框、三个提示信息的参数；利用一个函数 static void init_layout()封装界面所有五个按钮、三个提示信息和三个文本框的初始化布局参数。在 draw_field()函数中增加 isActive 和 isReadOnly 两个参数，分别表示文本框是否被激活和是否是可读状态，参数的具体定义如下：

变量名	对应控制
isActive	isActive=1，数据 1 对应的输入框激活 isActive=2，数据 2 对应的输入框激活 isActive=0，数据 1、2 对应的输入框未激活 **注意**：激活的输入框的边变为蓝色，并画出输入光标提示符；未激活的输入框的边为黑色。
isReadOnly	isReadOnly=1，表示对应文本框为只读属性 isReadOnly=0，表示对应文本框为非只读属性 注意：框体为只读属性时，框边颜色为灰色。否则根据 isActive 的值，为黑色或者蓝色

注意：此任务只给出了定义，具体的边框颜色显示在任务 5 中才体现。

（3）数据输出显示要求（即运行界面）

在界面上输出五个按钮、三个提示信息和三个文本框，其中要求按钮绘制功能封装在 draw_button(const Button* b)函数中；文本框绘制封装在 static void draw_field()函数。输出界面如下：

2. 能力要素分析

1) 掌握界面整体参数化布局
2) 掌握功能分类及函数表述与封装

3) 掌握函数实现批量处理的方法

3. 实现思路

（1）数据结构

同任务 2

（2）算法思路

利用函数完成界面的整体布局初始化(init_layout())，各个要素利用变量体现，便于函数批量处理；利用函数封装完成按钮显示(draw_button())；利用函数封装完成文本框显示及状态控制(draw_field())

（3）实现流程

Step1：在 main()中利用 init_layout()函数完成窗体参数的集中初始化

Step2：利用循环调用 draw_button()完成五个按钮的绘制

Step3：利用 outtextxy()函数显示三组提示信息

Step4：调用三次 draw_field()函数完成三个本文框的绘制

Step5：设置主循环等待输入

Step6：退出时用 closegraph()关闭窗口

4. 问题解析

1) 参数化布局所带来的好处

可以手写每个控件坐标，但后续调整维护成本高；采用参数化布局更利于迭代与教学，**参数化布局**实现效率更高，示例如下：

```
// 第一行按钮的起始位置、按钮尺寸bw&bh、按钮间距gap
    int startX = 20, startY = 20;
    int bw = 90, bh = 45, gap = 12;

// 5个按钮显示的文字
    const TCHAR* texts[5] = { _T("+"), _T("-"), _T("*"), _T("/"), _T("退出") };

    // 5个按钮填充颜色：前4个 ORANGE，退出按钮 WHITE
    COLORREF cols[5] = { ORANGE, ORANGE, ORANGE, ORANGE, WHITE };
```

- 按钮的参数化布局：

```
    // 逐个设置五个按钮的位置、大小、文字、颜色
    for (int i = 0; i < 5; i++) {
        opBtns[i].x = startX + i * (bw + gap);
        opBtns[i].y = startY;
        opBtns[i].w = bw;
        opBtns[i].h = bh;
        _tcscpy_s(opBtns[i].text, 16, texts[i]);  // 安全拷贝按钮文字
        opBtns[i].color = cols[i];
```

```
}
```

■ 文本框的参数化布局：

```
int fieldY = 120;            // 输入框所在的 y 坐标
int fx = 20, fw = 160, fh = 45, fgap = 20; // 起始 x，宽，高，间距

// fieldn = FieldRect{fx, fieldY, fw, fh}; 等
field1 = { fx, fieldY, fw, fh };
field2 = { fx + (fw + fgap), fieldY, fw, fh };
fieldR = { fx + 2 * (fw + fgap), fieldY, fw, fh };
```

5. 实现要点

1) 按钮 Button 及输入/输出文本框的定义及例化

按钮Button的结构体定义

```
typedef struct {
    int x, y, w, h;
    TCHAR text[16];
    COLORREF color;
} Button;
```

输入/输出文本框

```
typedef struct {
    int x, y, w, h;
} FieldRect;
```

例化：

```
static Button opBtns[5];
static FieldRect field1, field2, fieldR;
```

2) 利用函数进行窗体的参数化布局

通过一个函数（例如init_layout()）集中把窗体数据进行参数化布局，包括：

① 按钮的初始位置、间隔、长宽等信息

```
int startX = 20, startY = 20;
int bw = 90, bh = 45, gap = 12;
```

② 按钮的显示文字及颜色

```
const TCHAR* texts[5] = { _T("+"), _T("-"), _T("*"), _T("/"), _T("退出") };
COLORREF cols[5] = { WHITE, WHITE, WHITE, WHITE, WHITE };
```

③ 五个按钮的参数化相对位置关系

```
for (int i = 0; i < 5; i++) {
    opBtns[i].x = startX + i * (bw + gap);
    opBtns[i].y = startY;
```

```
        opBtns[i].w = bw;
        opBtns[i].h = bh;
        _tcscpy_s(opBtns[i].text, 16, texts[i]);
        opBtns[i].color = cols[i];
    }
```

④ 文本框的尺寸参数

```
    int fieldY = 120;
    int fx = 20, fw = 160, fh = 45, fgap = 20;
```

⑤ 文本框的参数化相对位置关系

```
    field1 = { fx, fieldY, fw, fh };
    field2 = { fx + (fw + fgap), fieldY, fw, fh };
    fieldR = { fx + 2 * (fw + fgap), fieldY, fw, fh };
```

3) 按钮显示函数 draw_button()

输入参数：按钮参数结构体

功能：完成五个按钮的窗体绘制

示例：

```
static void draw_button(const Button* b) {
    setfillcolor(b->color);
    fillroundrect(b->x, b->y, b->x + b->w, b->y + b->h, 8, 8);

    setlinecolor(BLACK);
    roundrect(b->x, b->y, b->x + b->w, b->y + b->h, 8, 8);

    settextcolor(BLACK);
    settextstyle(22, 0, _T("Consolas"));

    int tw = textwidth(b->text);
    int th = textheight(b->text);
    outtextxy(b->x + (b->w - tw) / 2, b->y + (b->h - th) / 2, b->text);
}
```

4) 文本框显示及状态控制函数 draw_field()

输入参数：文本框参数结构体

功能：完成三个文本框的窗体绘制

示例：

```
static void draw_field(const FieldRect* f, const TCHAR* content, int isActive,
int isReadOnly) {
    setfillcolor(WHITE);
    solidrectangle(f->x, f->y, f->x + f->w, f->y + f->h);

    if (isReadOnly) setlinecolor(RGB(140, 140, 140));
    else setlinecolor(isActive ? BLUE : BLACK);
```

```
        rectangle(f->x, f->y, f->x + f->w, f->y + f->h);

        settextcolor(BLACK);
        settextstyle(20, 0, _T("Consolas"));
        outtextxy(f->x + 8, f->y + 12, content);

        if (isActive && !isReadOnly) {
            int tx = f->x + 8 + textwidth(content);
            setlinecolor(BLUE);
            line(tx + 2, f->y + 8, tx + 2, f->y + f->h - 8);
        }
    }
```

窗体显示的功能提升：

通过设置状态控制变量，来判断某个输入文本框是否被激活，从而改变文本框的状态。

例如：设置isActive变量，激活时边框为蓝色，未激活是边框为黑色

```
        setlinecolor(isActive ? BLUE : BLACK);
        rectangle(f->x, f->y, f->x + f->w, f->y + f->h);
```

5) const 关键字

const 关键字并非一开始就存在于 C 语言中，是在 **C89（ANSI C / ISO C 9899:1989）** 标准中被正式引入的，这个版本也常被称为**ANSI C**（1989年发布）。

C89（ANSI C / ISO C 9899:1989）

const首次被纳入C语言标准，核心目的：

- ✧ 定义“只读”变量（注意：C 中的const变量并非真正的常量，只是“只读”，比如不能用来定义数组长度，这和 C++ 不同）；
- ✧ 支持函数参数的只读保护；
- ✧ 提升代码的可读性和安全性，明确标识“不应该被修改”的变量 / 指针。

声明形式	能否修改指针本身	能否通过指针修改指向的对象	核心作用
Button* b	☑ 可以	☑ 可以	无限制，普通指针
const Button* b	☑ 可以	✕ 不可以	保护指向的对象不被修改

任务 4：实现整界面重绘功能

1. 任务要求

（1）数据输入操作要求

无

（2）数据存储和处理要求

保留任务 3 中按钮绘制函数 draw_button(const Button* b)函数和文本框绘制函数 draw_field()，并在此基础上，将界面的所有绘制动作都封装在 static void draw_ui()函数中，以后在需要重新绘制界面时，直接调用此函数，从而简化了设计。

（2）数据输出显示要求（即运行界面）

调用 draw_ui()函数在界面上输出五个按钮、三个提示信息和三个文本框，其中要求按钮绘制功能时调用 draw_button(const Button* b)函数，文本框绘制时调用 void draw_field()函数。此任务只是整合了输出过程，为后续设计重新刷新界面奠定了函数基础，输出界面如下：

2. 能力要素分析

1) 掌握界面分层绘制顺序设计
2) 掌握重绘函数的封装与复用

3. 实现思路

（1）数据结构

同任务 3

（2）算法思路

将界面具体的绘制动作，都封装在 draw_ui()函数，这样在需要界面重绘时，直接调用此函数就可完成界面重绘任务，使得程序设计和控制更为清晰。

（3）实现流程

Step1：在 main()中利用 init_layout()函数完成窗体参数的集中初始化

Step2：调用 draw_ui()完成界面绘制

Step2.1：设置背景颜色 setbkcolor(WHITE)

Step2.2：清屏 cleardevice()

Step2.3：绘制五个按钮

Step2.4：设置字体颜色并显示三个提示词

Step2.5：分别绘制三个文本框

Step3：设置主循环等待输入

Step4：退出时用 closegraph()关闭窗口

4. 问题解析

1) 窗体整体重获的作用

在 EasyX 中，“窗体整体重绘”本质是通过 cleardevice()（清屏） + 重新绘制所有图形 / 文字 / 控件，让窗口内容刷新到最新状态。

窗体整体重绘的核心作用：

- **作用 1：消除图形“残影”，保证画面干净（最核心）**

  如果没有整体重绘，每次绘制新内容（比如移动的文字、状态变化的按钮）时，旧内容不会消失，会叠加在窗口上形成“残影”。

- **作用 2：同步更新所有元素的状态**

  图形窗口中的元素（按钮、文字、动画）往往是“联动”的，整体重绘能保证所有元素同时刷新到最新状态：

  比如你绘制按钮的代码，如果按钮颜色因点击改变，只有通过“清空窗口 → 重新画新颜色的按钮 → 重新画文字”，才能让按钮的新样式立刻生效，且和其他元素（比如按钮上的文字）保持同步。

- **作用 3：实现动画的“帧刷新”（动画场景必备）**

  动画的本质是“快速切换帧画面”，整体重绘就是每一帧的“刷新操作”：

  第 1 帧：清空窗口 → 绘制小球在 (100,100) 位置；

  第 2 帧：清空窗口 → 绘制小球在 (105,100) 位置；

  重复上述步骤，肉眼就看到小球在移动。

  没有整体重绘，小球的所有位置会叠加，变成一条“轨迹线”，而非流畅的移动。

- **作用 4：修复窗口异常（比如遮挡后恢复）**

  如果图形窗口被其他窗口遮挡，或者最小化后恢复，窗口内容可能会丢失 / 错乱。整体重绘逻辑（一般放在主循环里）会自动重新绘制所有内容，让窗口恢复正常显示。

统一重绘函数有助于减少代码重复，后续改样式时也更方便。

5. 实现要点

1) 重绘函数的封装

重绘函数主要包括的流程有：

清屏 -> 外框 -> 按钮 -> 标签 -> 三个文本框 -> 提示显示

示例：

```
static void draw_ui() {
    setbkcolor(WHITE);
    cleardevice();

    setlinecolor(BLACK);
    rectangle(10, 10, 550, 210);

    for (int i = 0; i < 5; i++) draw_button(&opBtns[i]);

    settextcolor(BLACK);
    settextstyle(20, 0, _T("Consolas"));
    outtextxy(field1.x, 90, _T("数据1"));
    outtextxy(field2.x, 90, _T("数据2"));
    outtextxy(fieldR.x, 90, _T("计算结果"));

    draw_field(&field1, buf1, activeField == 1, 0);
    draw_field(&field2, buf2, activeField == 2, 0);
    draw_field(&fieldR, bufR, 0, 1);
}
```

2) 重绘函数的调用

```
int main() {
  initgraph(560, 220);
  init_layout();
  draw_ui();   //调用重绘函数

 //主函数响应输入处理部分（略）

  closegraph();
  return 0;
  }
```

注意：后续任务中在窗体显示内容跟新时，也要进行窗体重绘函数调用。

任务 5：实现文本框焦点管理及消息循环与事件分发

1. 任务要求

（1）数据输入操作要求

程序能够接收鼠标左键按下输入(仅判断在本文框区域)和键盘按键(仅限 ESC 键）输入，其中鼠标左键按下输入调用相应的处理函数，键盘 ESC 直接退出程序。注意：任务鼠标输入只限制在对文本框的点击识别。

（2）数据存储和处理要求

程序利用 peekmessage()函数捕获系统的鼠标左键点击输入和 ESC 输入。若是 ESC 输入，则选择直接退出。

如果是鼠标左键点击输入，则调用 handle_mouse_down()加以处理。函数 handle_mouse_down()调用 in_rect()来进行鼠标点击区域判断。

若点击在数据 1 的文本框内，则置激活变量 activeField=1（激活）；

若点击在数据 2 的文本框内，则置激活变量 activeField=2（激活）；

若点击在计算结果文本框内，则置激活变量 activeField0 =0（未激活）。

（2）数据输出显示要求（即运行界面）

界面首次显示时，数据 1 和 2 的输入框边框颜色为黑色，计算结果框为灰色。当鼠标点击在数据 1 输入框时，输入框为蓝色，并画出输入光标；当鼠标点击在数据 2 输入框时，输入框为蓝色，并画出输入光标，此时数据 1 输入框边变为黑色；计算结果框始终为灰色。

注意：本任务只接收鼠标点击文本框并改变颜色，没有实现数据的输入功能。数据输入并回显功能在任务 6 中实现。

数据框被激活的状态变化，如图 5.1 和图 5.2 所示

图 5.1 数据 1 框被激活的状态，蓝色边框+输入光标

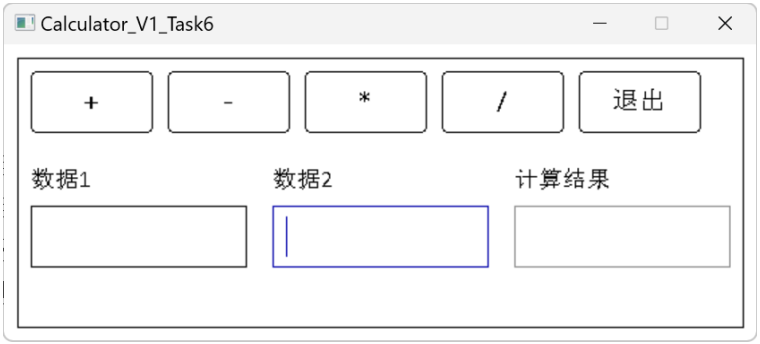


图 5.2 数据 2 框被激活的状态，蓝色边框+输入光标

2. 能力要素分析

1) 掌握鼠标点击事件的消息获取及处理机制
2) 掌握鼠标命中输入文本框的判断逻辑
3) 掌握输入文本框被激活后的处理方式
4) 掌握系统消息循环和事件分发机制

3. 实现思路

（1）数据结构

同任务 4

（2）算法思路

接收鼠标左键点击消息，并判断鼠标左键点击区域。若点到数据 1 对应的输入框，改变框体颜色为蓝色，并绘制输入光标；若点到数据 2 对应的输入框，改变框体颜色为蓝色，同时利用界面重绘将数据 1 对应的输入框颜色给为黑色。改变界面后要进行界面整体重绘。

（3）实现流程

Step1：在 main()中利用 init_layout()函数完成窗体参数的集中初始化

Step2：调用 draw_ui()完成界面绘制

Step3：定义 ExMessage 变量，并设置主循环等待输入

Step3.1：若输入鼠标左键，则调用 handle_mouse_down(msg.x, msg.y)处理

Step3.1.1 判断鼠标左键点击的区域，并设置相应的状态值 activeField

Step3.1.2 调用 draw_ui()函数整体重绘界面

Step3.2：若输入为 ESC 键，则 closegraph()关闭窗口，退出程序

4. 问题解析

1) 如何获取鼠标或键盘的动作消息

EasyX 在 graphics.h 中声明了 peekmessage 函数，核心作用是简化输入处理。

原型如下：

bool peekmessage(ExMessage *msg, DWORD filter = EM_MOUSE | EM_KEY | EM_CHAR);

参数说明：

- ExMessage *msg：EasyX 自定义的消息结构体，存储鼠标 / 键盘 / 字符输入信息；
- DWORD filter：筛选输入类型，支持 EasyX 定义的宏：
  - ✧ EM_MOUSE = 1：筛选鼠标消息；
  - ✧ EM_KEY = 2：筛选键盘按键消息；
  - ✧ EM_CHAR = 4 筛选字符输入消息；
  - ✧ 可通过|组合（如 EM_MOUSE | EM_CHAR），表示只捕获鼠标+字符输入。

ExMessage 结构体：

ExMessage 是 EasyX 图形库定义的**消息结构体**，专门用来存储窗口的交互事件（比如鼠标点击、键盘按键、鼠标移动等）。ExMessage *msg 是指向这个结构体的指针，方便在函数中传递或批量处理消息。

核心作用：**把用户的操作（点击、按键、移动）转化为程序能识别的“消息数据”**，是实现图形界面交互（比如按钮点击、鼠标拖拽）的基础。

定义：

```
struct ExMessage
{
    // 1. 消息类型（核心）：标识这是哪种操作
    UINT message;

    // 2. 鼠标相关参数（鼠标消息时有效）
    int x;          // 鼠标位置的 X 坐标
    int y;          // 鼠标位置的 Y 坐标
    bool ctrl;      // Ctrl 键是否按下
    bool shift;     // Shift 键是否按下
    bool lbutton;   // 鼠标左键是否按下
    bool mbutton;   // 鼠标中键是否按下
    bool rbutton;   // 鼠标右键是否按下
    short wheel;    // 鼠标滚轮滚动值（正数向上，负数向下）

    // 3. 键盘相关参数（键盘消息时有效）
    TCHAR vkcode;   // 虚拟键码（比如 VK_ESCAPE 代表ESC键）
    TCHAR ch;       // 按键对应的字符（比如按A键，ch='A'）
    TCHAR scancode; // 扫描码（底层硬件编码，新手几乎不用）

    // 4. 其他补充参数（极少用）
    DWORD time;     // 消息产生的时间
```

```
    HWND hwnd;        // 接收消息的窗口句柄
};
```

结构体成员的含义：

成员	类型	核心含义	常用场景
message	UINT	消息类型： - WM_MOUSEMOVE：鼠标移动 - WM_LBUTTONDOWN：鼠标左键按下 - WM_KEYDOWN：键盘按键按下 - WM_CHAR：键盘字符输入	所有交互逻辑的“判断入口”
x/y	int	鼠标位置坐标（窗口内）	鼠标点击按钮、拖拽等
lbutton	bool	鼠标左键是否按下	按钮点击检测
vkcode	TCHAR	键盘虚拟键码（系统宏定义），比如 VK_SPACE（空格）、VK_UP（上方向键）	键盘方向键、功能键检测
ch	TCHAR	按键对应的字符（如 '0'-'9'、'a'-'z'）	输入框文字输入

2) 如何判断 ESC 按键被按下

ESC 按键被按下后，系统反馈获得 WM_KEYDOWN 消息，但是这个消息获得信息还不够，要需要进一步判断按键类型，此时就用到了虚拟键码（VK_*）的概念。

消息类型	处理场景	核心原因
WM_KEYDOWN	控制键（ESC、方向键、F1-F12 等）	关注**物理按键**，无字符编码 / 大小写 / 输入法处理，适合控制类操作
WM_CHAR	字符输入（字母、数字、标点、小数点）	由 TranslateMessage 转换而来，包含**最终字符编码**，适合文本输入处理

VK_ESCAPE 的含义：

VK_ESCAPE 是 Windows 系统定义的虚拟键码（Virtual Key Code），本质是一个代表键盘上「ESC 键」的数值常量（实际值为 0x1B，即十进制的 27）。

VK：是 Virtual Key 的缩写，意为“虚拟键”——Windows 用一套统一的数值来标识键盘上的物理按键，和具体的字符编码、输入法无关；

ESCAPE：对应键盘上的 ESC 键（退出键 / 取消键），是最常用的控制键之一。

5. 实现要点

1) 利用 peekmessage()获取鼠标及键盘动作的消息

```
ExMessage msg;
 while (1) {
     while (peekmessage(&msg, EM_MOUSE | EM_KEY)) {
         if (msg.message == WM_LBUTTONDOWN) {
             handle_mouse_down(msg.x, msg.y);
         }
         else if (msg.message == WM_KEYDOWN && msg.vkcode == VK_ESCAPE) {
             closegraph();
             return 0;
         }
     }
     Sleep(10);
 }
```

其中：

- ✧ WM_LBUTTONDOWN 表示鼠标左键按下消息，对应的处理函数是：handle_mouse_down(&msg)；
- ✧ WM_KEYDOWN 表示有控制键按下，需要再结合 VK_ESCAPE 来确定是否按下 ESC 退出键，对应的处理函数是：closegraph()

2) 获得鼠标动作消息后，如何获取鼠标点击的坐标

可以在自编函数 handle_mouse_down(msg.x, msg.y) 中利用结构体 msg 中的坐标信息（具体定义见上表）。

调用方式如下：

```
while (peekmessage(&msg, EM_MOUSE | EM_KEY)) {
            if (msg.message == WM_LBUTTONDOWN) {
                handle_mouse_down(msg.x, msg.y);
```

入口参数只需要鼠标的位置信息 x,y

鼠标动作处理函数如下：

```
static void handle_mouse_down(int mx, int my) {
    if (in_rect(mx, my, field1.x, field1.y, field1.w, field1.h)) {
        activeField = 1;
    }
```

```
    else if (in_rect(mx, my, field2.x, field2.y, field2.w, field2.h)) {
        activeField = 2;
    }
    else if (in_rect(mx, my, fieldR.x, fieldR.y, fieldR.w, fieldR.h)) {
        activeField = 0;
    }
    else {
        activeField = 0;
    }
    draw_ui();
}
```

3) 如何判断是否点击到指定全区域

```
// ----------------------------
// 工具函数：判断点是否在矩形内
// ----------------------------
static int in_rect(int mx, int my, int x, int y, int w, int h)
{
    // 判断鼠标坐标 (mx,my) 是否落在矩形区域 [x, x+w] × [y, y+h] 内
    return (mx >= x && mx <= x + w && my >= y && my <= y + h);
}
```

4) 如何控制文本框的激活状态

可以采用静态全局变量activeField来表示文本框的激活状态。定义逻辑如下：

- ✧ 未激活，activeField = 0
- ✧ 文本框1激活，activeField = 1
- ✧ 文本框2激活，activeField = 2

全局变量定义时默认值为未激活状态

```
static int activeField = 0;
```

示例：

```
if (in_rect(mx, my, field1.x, field1.y, field1.w, field1.h)) {
        activeField = 1;
    }
    else if (in_rect(mx, my, field2.x, field2.y, field2.w, field2.h)) {
        activeField = 2;
    }
    else if (in_rect(mx, my, fieldR.x, fieldR.y, fieldR.w, fieldR.h)) {
        activeField = 0;  //这是输出框，始终为未激活状态
    }
    else {
        activeField = 0;
    }
```

任务 6：实现键盘字符输入

1. 任务要求

（1）数据输入操作要求

本任务只是完成在数据 1 和数据 2 输入框中实现数据输入并回显，回显时利用界面整体重绘函数来改变输入框的颜色。键盘输入新增加对退格键的处理。

（2）数据存储和处理要求

采用字符数组的形式来定义数据 1/2 的输入，以及计算结果本文框的输出，具体定义如下：

```
static TCHAR buf1[32] = _T("");
static TCHAR buf2[32] = _T("");
static TCHAR bufR[64] = _T("");
```

输入数据采用字符形式，在具体计算时，再转换为数值形式。

若点击在数据 1 的文本框内，则置激活变量 activeField=1（激活）；

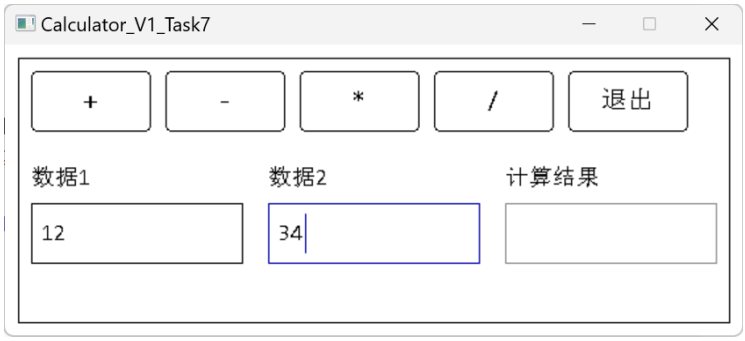
若点击在数据 2 的文本框内，则置激活变量 activeField=2（激活）；

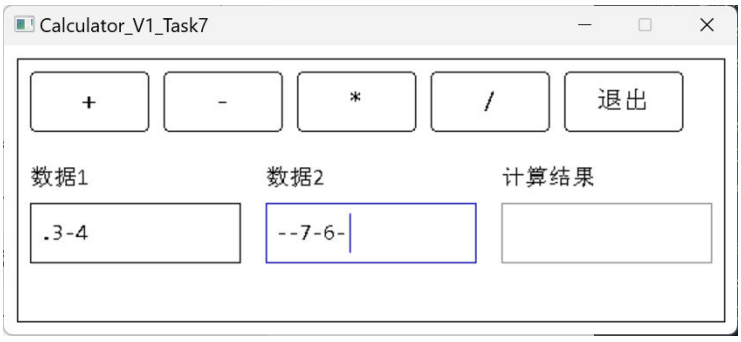
若点击在计算结果文本框内，则置激活变量 activeField0 =0（未激活）。

（2）数据输出显示要求（即运行界面）

根据数据的输入，利用界面整体重绘函数来改变输入框的颜色。

程序运行输出结果如下：





2. 能力要素分析

1) 掌握键盘字符输入消息的获取处理（WM_CHAR）
2) 掌握文本缓冲区的追加或删除操作
3) 掌握焦点相关输入控制

3. 实现思路

（1）数据结构

在任务 5 数据结构的基础上，增加文本框内容的字符数组，如下：

static TCHAR buf1[32] = _T("");
static TCHAR buf2[32] = _T("");
static TCHAR bufR[64] = _T("");

（2）算法思路

接收鼠标左键点击消息，并判断鼠标左键点击区域。若点到数据 1 对应的输入框，改变框体颜色为蓝色，并绘制输入光标，接收键盘数据输入并回显；若点到数据 2 对应的输入框，改变框体颜色为蓝色，接收键盘数据输入并回显，同时利用界面重绘函数，将数据 1 对应的输入框颜色改为黑色。若接收到退格键，则进行删除动作，并刷新界面。

（3）实现流程

Step1：在 main()中利用 init_layout()函数完成窗体参数的集中初始化
Step2：调用 draw_ui()完成界面绘制
Step3：定义 ExMessage 变量，并设置主循环等待输入
Step3.1：若有鼠标左键点击输入，则调用 handle_mouse_down(msg.x, msg.y)处理
Step3.1.1 判断鼠标左键点击的区域，并设置相应的状态值 activeField
Step3.1.2 调用 draw_ui()函数整体重绘界面
Step3.2：若键盘输入，则调用 handle_char_input((TCHAR)msg.ch)函数
Step3.2.1 若输入为 0-9 数字、小数点或者减号，则回显，并重绘界面
Step3.2.2 若输入为退格键，则调用退格键处理函数 backspace_active()，并重绘
Step3.3：若输入为 ESC 键，则 closegraph()关闭窗口，退出程序

4. 问题解析

1) 如何获取系统的键盘输入消息

```
ExMessage msg;
while (1) {
    while (peekmessage(&msg, EM_MOUSE | EM_KEY | EM_CHAR)) {
        if (msg.message == WM_LBUTTONDOWN) {
            handle_mouse_down(msg.x, msg.y);
        }
        else if (msg.message == WM_CHAR) { //获取键盘输入的信息
```

```
            handle_char_input((TCHAR)msg.ch);
        }
        else if (msg.message == WM_KEYDOWN && msg.vkcode == VK_ESCAPE) {
            closegraph();
            return 0;
        }
    }
```

利用 WM_CHAR 处理字符输入。

注意：不要使用 WM_KEYDOWN 来处理字符输入，原因如下：

- WM_KEYDOWN：当用户按下任意键盘按键时触发，它关注的是物理按键（比如按下 'A' 键，无论是否按住 Shift），消息参数是虚拟键码（VK_*），不处理字符的大小写、输入法等。
- WM_CHAR：是在 WM_KEYDOWN 之后，由系统处理了按键的上下文（Shift 键、输入法、大小写锁定等）后生成的字符消息，它传递的是最终的字符编码（比如按下 Shift+A 会产生 'A'，仅按 A 会产生 'a'）。

由于 WM_CHAR 是 WM_KEYDOWN 的“加工后产物”，如果同时在这两个消息的处理逻辑中追加字符，相当于对同一个输入操作执行了两次字符追加，自然会造成重复输入。

5. 实现要点

1) 字符输入处理函数 handle_char_input((TCHAR)msg.ch)

函数功能：

（1） 判断两个输入框是否处在激活状态，如果都否，则直接退出；

（2） 判断是否是退格键（c == 8），如是则调用退格处理函数backspace_active()

（3） 判断是否为允许的输入字符，如是则调用字符显示函数 append_char_to_active(c)；

（4） 每次调用函数后，都要重新整体绘制界面

示例：

```
static void handle_char_input(TCHAR c) {
    if (activeField != 1 && activeField != 2) return;

    if (c == 8) {
        backspace_active();
        draw_ui();
        return;
    }

    if ((c >= _T('0') && c <= _T('9')) || c == _T('.') || c == _T('-')) {
```

```
            append_char_to_active(c);
            draw_ui();
        }
    }
```

2) **退格处理函数 backspace_active()**

```
// ----------------------------
// 对当前激活输入框执行退格
// ----------------------------
static void backspace_active() {
    if (activeField != 1 && activeField != 2) return;
    TCHAR* target = (activeField == 1) ? buf1 : buf2;

    size_t len = _tcslen(target);
    if (len == 0) return;

    // 删除最后一个字符
    target[len - 1] = _T('\0');

}
```

3) 字符显示输出函数 append_char_to_active()

```
static void append_char_to_active(TCHAR c) {
    if (activeField != 1 && activeField != 2) return;

    TCHAR* target = (activeField == 1) ? buf1 : buf2;
    size_t len = _tcslen(target);
    if (len >= 31) return;

    target[len] = c;
    target[len + 1] = _T('\0');
}
```

任务 7：实现四则运算与结果输出

1. 任务要求

（1）数据输入操作要求

在任务 6 基础上，增加对五个按钮的鼠标左键输入的判断，并完成相应的动作。

（2）数据存储和处理要求

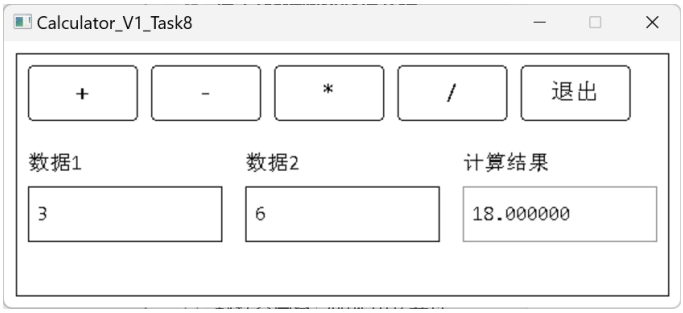
在鼠标处理函数 handle_mouse_down(int mx, int my)中，增加对鼠标左键点击按钮的逻辑判断，并调用相应的处理函数。如果点击“退出”按钮，则关闭界面，结束退出；若点击加、减、乘、除按钮，则调用 do_calc(opBtns[i].text[0])加以计算并显示，然后重绘界面。

（3）数据输出显示要求（即运行界面）

在任务 6 的功能基础上，增加了对输入数据的加、减、乘、除运算，并将结算结果输出到“计算结果”输出框内，然后对界面进行整体重绘更新。

特别注意：对于数据 1 和数据 2 的每次修改完成后，都要马上清除“计算结果”框的内容，以实现显示同步。

程序运行结果如下：



2. 能力要素分析

1) 掌握 EasyX 的按钮触发判断方法；
2) 掌握将输入缓冲区的字符内容转为浮点数值的方法；
3) 掌握加减乘除四则运算的实现方法；
4) 掌握错误处理与用户提示；

3. 实现思路

（1）数据结构

同任务 6 数据结构。

（2）算法思路

接收鼠标左键点击加、减、乘、除及退出的按钮动作判断，点击“退出”按钮则直接退出程序；点击加、减、乘、除按钮，则调用 do_calc()函数进行运算并显示运算结果，然后进行界面重绘。

（3）实现流程

Step1：在 main()中利用 init_layout()函数完成窗体参数的集中初始化

Step2：调用 draw_ui()完成界面绘制

Step3：定义 ExMessage 变量，并设置主循环等待输入

Step3.1：若有鼠标左键点击输入，则调用 handle_mouse_down(msg.x, msg.y)处理

Step3.1.1 判断鼠标左键点击区域，若点击数据 1 或数据 2 对应的文本框，则完成相应的数据输入或退格处理，并重绘界面

Step3.1.2 若点击加减乘除按钮，在进行运算并显示运算结果，同时重绘界面

Step3.1.3 若点击“退出”按钮，则直接退出

Step3.2：若有键盘输入，则调用 handle_char_input((TCHAR)msg.ch)函数

Step3.2.1 若输入为 0-9 数字、小数点或者减号，则回显，并重绘界面

Step3.2.2 若输入为退格键，则调用退格键处理函数 backspace_active()，并重绘

Step3.3：若输入为 ESC 键，则 closegraph()关闭窗口，退出程序

4. 问题解析

1) “计算结果”输出框采用字符串形式的优点

结果缓冲区 bufR 使用字符串形式，既能显示正常数值，也能统一显示错误信息（如除零错误）。

采用这种统一输出的方式，具有如下特点：

- 定义字符串类型的结果缓冲区： bufR 作为统一的结果缓冲区，替代原本可能的数值类型变量；
- 统一的结果展示逻辑：无论是正常计算结果还是错误信息，都写入这个字符串缓冲区，再统一绘制/输出；
- 异常场景处理：在除法等易出错的计算逻辑中，先判断异常（如除数为 0），若触发则将错误信息写入 bufR；若正常则将数值转为字符串写入 bufR。

2) 五个功能按键点击触发的判断方式

在 EasyX 模式下，功能按钮的点击触发判断依旧采用鼠标几何位置判断式，只是在原有的 handle_mouse_down(int mx, int my)函数中新增加了对五个功能按钮的判断逻辑，具体实现方式如下：

```
static void handle_mouse_down(int mx, int my) {
    ……
    for (int i = 0; i < 5; i++) {
        if (in_rect(mx, my, opBtns[i].x, opBtns[i].y, opBtns[i].w, opBtns[i].h)) {
```

```
            if (_tcscmp(opBtns[i].text, _T("退出")) == 0) {
                closegraph();
                ExitProcess(0);
            }
            do_calc(opBtns[i].text[0]);
            draw_ui();
            return;
        }
    }
```

5. 实现要点

1) 数据 1 和数据 2 输入框的数据结构为字符型数组，计算前要转换成浮点数

利用_tstof( )函数将字符串转换为浮点数

```
// 将字符串转换为 double
// _tstof 会根据 TCHAR 版本选择 atof / _wtof 等
double a = _tstof(buf1);
double b = _tstof(buf2);
```

程序在初始化阶段都定义为字符型：

```
static TCHAR buf1[32] = _T("");
static TCHAR buf2[32] = _T("");
static TCHAR bufR[64] = _T("");
```

2) 四则运算函数的实现

通过按钮功能判断后，按加、减、乘、除四个按钮后就调用该函数。

函数输入参数：四种运算符。函数首先把两个输入框的信息由字符型转换为浮点数，然后根据输入的操作符进行运算。其中，设置了除零错误和运算符错误两类异常操作的检测。计算完成后，将计算结果以字符形式输出。

示例：

```
static void do_calc(TCHAR op) {
    if (_tcslen(buf1) == 0 || _tcslen(buf2) == 0) {
        set_result_error(_T("请输入数据"));
        return;
    }

    double a = _tstof(buf1);
```

```
    double b = _tstof(buf2);
    double r = 0.0;

    switch (op) {
//四则运算功能
    }

    _stprintf_s(bufR, 64, _T("%.6f"), r);
}
```

3) 统一的错误输出机制

为了规范化操作，增加了一个统一的错误信息输出函数，从而使程序流程变得更为清晰。

错误输出函数定义：

```
static void set_result_error(const TCHAR* msg) {
    _tcscpy_s(bufR, 64, msg);
}
```

调用形式：

```
set_result_error(_T("除零错误"));
set_result_error(_T("运算符错误"));
```

任务 8：程序健壮性处理与交互优化

1. 任务要求

（1）数据输入操作要求

无

（2）数据存储和处理要求

针对防御式编程，在程序中增加对输入长度限制（以防止字符数组越界）、加入空输入、除零、运算符错误等错误判断及提示，并利用“计算结果”文本框进行输出提示。

（2）数据输出显示要求（即运行界面）

各类错误提示，利用“计算结果”文本框进行输出。。

程序运行结果如下：

2. 能力要素分析

1) 掌握防御式编程思想。
2) 掌握各种异常操作的判断及处理方式；

3. 实现思路

（1）数据结构

利用逻辑判断，确定各种异常情况，使程序具有更好的鲁棒性。

（2）算法思路

在字符数组处理时，增量了数组长度限制和判断；对小数点的输入进行了约束，小数点只能一个；对输入的负号加以约束，负号只能首位；每次输入框内容修改后，应立刻清空运算结果输出框内容

（3）实现流程

Step1：在四则运算 do_calc()中增加对两个数据输入框的长度判断，若有一个为空，则提示“请输入数据”

Step2：四则运算 do_calc()中增加除法时增加分母为零检测，若为零则输出“除零错误”

Step3：在字符回显 append_char_to_active()函数中，增加对小数点、负号的异常操作检验与限制

4. 问题解析

1) 防御式编程思想

防御式编程（Defensive Programming）核心是：假设程序运行中会遇到各种“意外”（比如用户输入错误、参数异常、硬件 / 环境问题），提前在代码中做好“防护措施”，让程序即使遇到意外也不会崩溃，而是优雅处理。

✧ 普通编程：默认用户/环境都会按预期来（比如默认鼠标点击一定在窗口内、按钮坐标一定合法）；

✧ 防御式编程：默认用户/环境可能出各种问题，提前预判并处理（比如先判断鼠标坐标是否超出窗口范围，再处理按钮点击）。

防御式编程的核心原则：

✧ **原则 1：参数 / 数据合法性校验（最常用）**

对所有输入、参数、变量做“有效性检查”，避免非法值导致程序崩溃。

✧ **原则 2：异常情况的“兜底处理”**

遇到意外时，不崩溃，而是给出提示/降级处理。

✧ **原则 3：限制操作边界，避免越界**

比如处理鼠标消息时，先判断鼠标是否在窗口内，再处理按钮点击：

✧ **原则 4：资源的“安全释放”**

5. 实现要点

1) 对输入设置检查缓冲区长度功能，防止数组越界

示例：

```
TCHAR* target = (activeField == 1) ? buf1 : buf2;
    size_t len = _tcslen(target);
    if (len >= 31) return;
```

2) 对小数点与负号进行轻量约束，减少非法字符串转换问题

约束逻辑：

- 负号（-）：只能出现在输入字符串的第一个位置，且只能出现一次。

  示例：

```
//负号只能在首位
if (c == _T('-')) {
    if (len != 0) return;
}
```

- 小数点（.）：整个字符串中只能出现一次，且不能单独作为输入内容，也不能出现在负号之后、数字之前（如 -.123 这种格式也可接受，属于合法数字格式）。

  示例：

```
if (c == _T('.')) {
    //1)只能有一个小数点
```

```
        for (size_t i = 0; i < len; i++) {
            if (target[i] == _T('.')) return;
        }

        //2）不能单独作为第一个字符输入，即不允许直接输入
        if (len == 0) return;

        //3) 如果当前是 "-"，允许输入 "."
        //   因为 "-.123" 是合法格式
        //   所以这里不用额外禁止 len==1 && buf[0]==
    }
```

- 过滤掉除数字、负号、小数点外的所有其他字符。

```
    if ((c >= _T('0') && c <= _T('9')) || c == _T('.') || c == _T('-')) {
        append_char_to_active(c);
        draw_ui();
        return;
    }
```

3) 每次输入框内容修改后，应立刻清空运算结果输出框内容

每次输入框有内容更新后，都要调用clear_result_on_edit()函数，将运算结果输出框的内容及时清理，以保证显示内容的一致性。

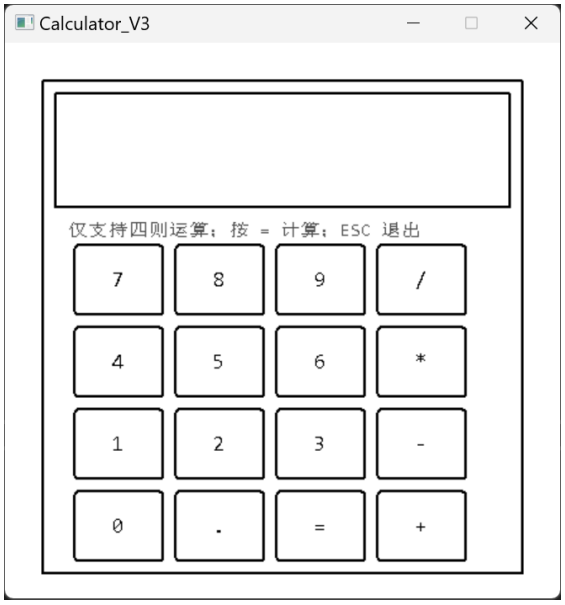
函数定义如下：

```
static void clear_result_on_edit() {
    bufR[0] = _T('\0');
}
```

完整的应用示例：

```
static void append_char_to_active(TCHAR c) {
    if (activeField != 1 && activeField != 2) return;

    TCHAR* target = (activeField == 1) ? buf1 : buf2;
    size_t len = _tcslen(target);
    if (len >= 31) return;

    if (c == _T('.')) {
        //1)只能有一个小数点
        for (size_t i = 0; i < len; i++) {
            if (target[i] == _T('.')) return;
        }

        //2）不能单独作为第一个字符输入，即不允许直接输入
        if (len == 0) return;
```

```
        //3) 如果当前是 "-"，允许输入 "."
        //   因为 "-.123" 是合法格式
        //   所以这里不用额外禁止 len==1 && buf[0]==

    }

    //负号只能在首位
    if (c == _T('-')) {
        if (len != 0) return;
    }

    target[len] = c;
    target[len + 1] = _T('\0');
    clear_result_on_edit();
}
```

思考题

项目也可进行 GUI 界面优化和功能提高，可以完成如下图所示的计算器。